PersonaForge: Agent-to-Agent Package Project Report

THE PROBLEM

The Agent-to-Agent Package is a terminal app that lets users create personas and then have them talk to each other about a topic of choice. The starter code contains the agent's instructions in its persona file plus a set of in-character rules.

The router is what decides what the user meant. The starter version did that with regular expressions, fixed text patterns it matched against what the user typed. For example, a pattern required that the user had to type sentences shaped like "create a persona …". The starter router could also only do one action per message input, so if the user prompted the app to list the personas and then have them talk in the same message, the app would only list the personas and drop the second request.

Tested sentences on the starter version:

What I typed	Expected	What happened	Why
create a persona	asks what kind	The same pre-written reply	hard-coded empty check, not a real question
make two doctors and a nurse	creates 3 personas	Offline placeholder reply ("Im a stub")	no word "persona", regex misses
list the personas then have them talk about diet	list, then talk	only listed	first regex match wins
I want a nurse character	creates a nurse	Offline placeholder reply ("Im a stub")	no word "persona", regex misses

These sentences were tested in offline mode as with the real model; a sentence the router doesn't recognize gets a chat reply instead from Claude, but still no personas or conversation would be created.

This problem matters because users actually trying to use the app have to learn hidden rules the app never tells them. For example, a gym manager who wants to rehearse a conversation with a member who's thinking of cancelling and types *"make a trainer and a member who wants to cancel"* gets no personas back, because they didn't follow the needed pattern to create a persona.

MY APPROACH & WHY

I replaced the regex router with tool use. Instead of guessing what the user meant, Claude picks an action from a provided menu, and the code carries it out. This section covers why I chose it, how it works, the design decision behind it, and which model I used.

TOOL USE

With tool use, Claude reads the user's sentence along with a menu of actions the app can do, then picks which ones to use and fills in the details, allowing the user to more accurately get what they asked for, improving upon the issue of the user having to know the hidden rules. Tool use was picked as the priority to improve the app because it would be the most valuable improvement.

HOW IT WORKS

The menu, *TOOLS*, is a list of the three actions Claude can ask for (create a persona, list personas, start a conversation), each with a description and the details Claude has to fill in. *complete_with_tools* is the communicator; it sends the user's message and the menu to Claude and brings back Claude's reply, including its request. *_run_tool* takes a request and runs the matching function; The loop asks Claude, gets its requests, and sends the results back, repeatedly until Claude is done. Claude only chooses; the code performs the work. For example, "make two personal trainers and a client" comes back as three create_persona requests; the loop creates three personas, and Claude confirms them.

DESIGN

- The starter regex router was kept as an offline fallback and as the baseline to compare against. The app checks whether the model can use tools. Claude can, so it uses tool use. Offline omde and the test models can't, so they use the old router.
- I implemented a 5-round cap, so Claude gets at most 5 back-and-forths per message. A normal request takes 2 or 3. The cap makes sure the app stops after 5 and tells the user, instead of looping and running costs.
- Set a 14-turn cap on conversations to bound API costs, since every line is one API call. The default is 6, and 14 is enough to have 7 lines each per two personas.
- Memory was also implemented for the router: it remembers the last 10 messages (5 back-and-forths) as plain text, so after creating the desired personas, the user can refer to them as "them" and the app understands who the user means. Capping it at 10 keeps each request from resending the whole session.
- Told Claude to ask clarifying questions when something is missing instead of guessing, like "create persona" with no description.
- Failures don't crash the app; a blank message gets an answer right away, a failed call to Claude shows the error type instead of crashing, a conversation that stops midway

through keeps the lines already said, and incorrect turn values such as “8 (as a string)” or -4 get turned into the correct number.

MODEL CHOICE

While building on the starter structure, Haiku was used to test and validate improvements and design choices. Haiku was chosen because of its lower cost and speed, since building is an interactive process. The app’s default is Claude Opus 5, the most capable and most expensive option.
During the concluding workflow of the project, a test between Haiku and Sonnet was run to compare how well each followed the agents’ instructions, using the same personas and request, three runs each. The results are in the *NARRATION* section. Based on that test, I decided to change the app’s default model from Opus to Sonnet. Sonnet followed the agents’ instructions reliably and at a lower cost than Opus. The final transcripts were produced by Sonnet-5.

TESTING

I tested the app in two ways: offline tests that check the code, and live runs with the real model that checked how Claude behaves.

OFFLINE TESTS
The starter code provided 41 tests, and there are now 61 total tests. The offline tests can't use Claude because Claude’s answers would differ every run, and it would also cost money to do so. Instead, the test uses a fake model that plays back replies ahead of time. This would allow me to check that the app would perform the correct instruction with what Claude sends back, e.g., a request to “create a persona” actually creates one, and if Claude sends a -6 for the number of returns, the app turns it into a number that it can actually use.

Test Type	What the tests check
Tool Use	Normal sentence creates a persona Several actions in one message Ask clarifying question Doesn’t crash under unknown action Persona list is shown to user
Caps	Stops after 5 rounds 14 turns Bad turn values become usable
Memory	Previous message is remembered Claude is told what personas exist

Personas	Same names don't overwrite
Conversations	Agents are told the topic and who's involved Stage directions removed, only dialogue
Errors	Blank message isn’t sent to Claude Failed API call doesn’t crash the app

LIVE TESTING

Since offline tests can only check that the code handles whatever Claude sends back and not Claude’s actual behavior, I ran the app live and saved the transcripts to determine what failed with the real model. This brought problems that weren’t visible in offline testing. For example, the characters the user creates (the agents) were confused about who they were talking to. In one run, a doctor said, “*I can’t play Dr. Marcus Chen”* because the character thought that another character was telling it to play a different role. In another run, after the conversation had concluded, Claude showed the user a recap of a conversation that didn’t occur.

WHAT LIVE TESTING FOUND

AGENTS DIDN'T KNOW WHO WAS IN THE ROOM

As stated above, during live testing, agents didn’t know who was in the room, which led to the example error of the doctor stating, *“I can’t play Dr. Marcus Chen*.” This means that each persona created had no context of who they were going into a conversation with. There was also a fake speaker present that would state *“Scene: diet”* in a way to inform the characters what the conversation should center around, but paired with the lack of context of who else was involved, it led to more confusion. The agents read that line as someone talking to them. The first reply was *“I’m not quite sure what you mean by scene: diet”*. I addressed this by telling each agent the situation and who else was in the conversation (their name, role, and a one-line summary) as part of the instructions, and by removing the fake Scene speaker. I also improved the tool menu so that Claude sends a real-world situation about the topic in a way to provide the agents proper context, instead of just a single word. Following the improvement, the next live run saw the agents refer to each other by name and build upon each other’s points.

NAMES COLLIDING AND FILES OVERWRITING

I also encountered that the app would overwrite persona files. The user could have made 6 personas, but the app would have 4 persona files on disk, meaning two got overwritten. This would occur because Claude would give colliding names since each persona is written in a separate call that can’t see the others, and the app would overwrite the first persona with the newest one containing the same name. This was addressed in two steps. First, *generate_persona*

(the persona writer) is now told which names are already taken, so it picks a new one. Second, if Claude were to still repeat a name, *save_persona* (saving step) adds a number to the filename, like *marcus-chen-2.md*, instead of overwriting the first file.

THE ROUTER FORGOT THE LAST MESSAGE

If a user were to prompt the app to *"have them talk"*, the app would not understand who *"them"* refers to because every message starts from scratch, carrying no memory from before. The app would reply with *"do you already have personas created?"*, while the personas were created in the input before. This was fixed by having the router remember the last 10 messages and telling it which personas exist, so *"them"* has a reference point.

THE MADE-UP RECAP

At the end of the conversation, Claude would provide a recap of the conversation. As well as the recap being unnecessary, it was also recapping a conversation that didn't take place. For instance, after one real conversation, it said *"Here's their conversation:"* and showed different lines about Mediterranean diets that none of the characters mentioned. This happened because Claude never saw the real lines. When a conversation ends, Claude only gets back a short note including the number of turns and the personas involved, so when it tried to recap, it made up a new conversation. The user ended up seeing two different conversations.

I fixed this by adding to the router's instructions: do not summarize, quote, or repeat the conversation as the user has already read every line; instead, suggest a next step. Following the improvement, the subsequent live runs since then have not made up recaps; Claude prompted the user to try a different scenario.

NARRATION: HAIKU VS. SONNET

During the workflow when Haiku was being used as the model, narration was a repeated occurrence, like *"'pauses', I lean against the table"*. The agents' instructions already stated not to include stage directions, no asterisks, no narration; however, Haiku seemed to ignore them. I fixed this in two steps. First, the code now removes stage directions written in asterisks. Second, I also added to the agents' rules an example of a correct line and a wrong one, instructing it to write only the words the personas would say out loud.

A small test was run to see the behavior between Haiku and Sonnet; the test included the same personas, requests, and 6 turns.

Narration Lines:

Run	Haiku	Sonnet
1	0/6	0/6
2	0/6	0/6
3	2/6	0/6
Total	**2/18 (11%)**	**0/18 (0%)**

Before I improved upon the agents' rules, Haiku narrated in almost every line. After the adjustments, Haiku narrated in 2 of 18 lines and Sonnet in none. The rewritten rules significantly improved Haiku performance, and Sonnet followed the instructions more reliably. Due to this small test, I decided to proceed with Sonnet as the default model for the app.

FAILURE ANALYSIS

BREAKING CHARACTER

Under the Sonnet model, I didn't see an agent break character in the 26 lines I tested. Haiku, on the other hand, after rule improvements, still saw agents breaking character, with nothing in the code detecting if it happens.

LOSING THE THREAD

Conversations stop at a fixed cap of turns, neglecting the fact that anything was decided or not in the conversation. Nothing in the code knows when the conversation is appropriately done. All six 6-turn tests ended mid-conversation, while the 8-turn showcase transcript did reach an appropriate ending because it had room to, not because anything in the code knew that the conversation had appropriately ended. Agents don't have memory between conversations. They only see their persona file and the current conversation. For instance, if personas were to make a deal and come back in 30 days, they wouldn't remember the deal made.

OTHER LIMITS

- Sonnet is the middle ground between Haiku and Opus, and Haiku performed relatively well after rule adjustments.
- There's a real risk for health or fitness use as the app makes confident health claims without nothing checking whether a health claim said by a character is true.
- Loose name matching: *find_persona* takes the first persona whose name matches what a user typed. For instance, if a user asked for *"Marcus"*, the function will retrieve the first

Marcus alphabetically without making sure the user gets the correct Marcus they wanted if there are two personas with the same first name but different last names.

FUTURE WORK

A future step involves an observer agent that would be able to read and check conversations in order to find agents breaking out of character, understanding that the conversation makes sense, e.g., whether a situation was addressed so that the conversation can end naturally. The observer agent can also flag assumptions stated as facts during agents' conversation, such as a personal trainer persona making a health claim as a stated fact and it not being verified.